In [ ]:
```
import sys
sys.version
```

python operators

Arithmatic operator

In [1]:
```
x1, y1 = 10, 5
```

In [2]:
```
x1 + y1
```

Out[2]: 15

In [3]:
```
x1 - y1
```

Out[3]: 5

In [4]:
```
x1 * y1
```

Out[4]: 50

In [5]:
```
x1 / y1  # float division
```

Out[5]: 2.0

In [6]:
```
x1 % y1
```

Out[6]: 0

In [7]:
```
x1 // y1   # int division
```

Out[7]: 2

In [8]:
```
x2 =3
y2 =3

x2 ** y2
```

Out[8]: 27

Assignment oprator

In [9]:
```
x = 2
```

In [10]:
```
x = x + 2
```

In [11]:
```
x
```

Out[11]: 4

In [12]:
```
x += 2
x
```

Out[12]: 6

In [13]:
```
x += 2
x
```

Out[13]: 8

In [14]:
```
x *= 2
x
```

Out[14]: 16

In [15]:
```
x -= 2
x
```

Out[15]: 14

In [16]:
```
x = 14
x
```

Out[16]: 14

In [17]:
```
x //=2
x
```

Out[17]: 7

In [18]:
```
x /= 2
x
```

Out[18]: 3.5

In [19]:
```
x2 = 14
x2 %=2
x2
```

Out[19]: 0

Relational Operator

In [20]:
```
a = 5
b = 6
```

In [21]:
```
a>b
```

Out[21]: False

In [22]:
```
a<b
```

Out[22]: True

In [23]:
```
a == b
```

Out[23]: False

In [24]:
```
a != b
```

Out[24]: True

In [25]:
```
b = 5
```

In [26]:
```
a == b
```

Out[26]: True

In [27]:
```
a>b
```

Out[27]: False

In [28]:
```
a>=b
```

Out[28]: True

In [29]:
```
a<=b
```

Out[29]: True

In [30]:
```
a!=b
```

Out[30]: False

Unary operator -----negative values only

In [31]:
```
x = 5
y = -x  # y becomes -5
z = -y  # z becomes 5
```

In [32]:
```
x = -5
y = +x  # y remains -5
```

In [33]:
```
x = 8
y = -x
z = -y
print(z)
print(y)
```

```
8
-8
```

Logical Operator

In [34]:
```
a = 5
b = 4
```

In [35]:
```
a<8 and b<5
```

Out[35]: True

In [36]:
```
a<8 and b<2
```

Out[36]: False

In [37]:
```
a<8 or b<2
```

Out[37]: True

In [38]:
```
a>8 or b<2
```

Out[38]: False

In [39]:
```
x = False
x
```

Out[39]: False

In [40]:
```
not x
```

Out[40]: True

In [41]:
```
x = not x
x
```

Out[41]: True

String Indexing

Forward indexing

In [42]:
```
s1 = 'welcome'
s1
```

Out[42]: 'welcome'

In [43]:
```
s1[0]   # forward index
```

Out[43]: 'w'

In [44]:
```
s1[4]
```

Out[44]: 'o'

In [45]:
```
s1[10]   # string not have that index
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[45], line 1
----> 1 s1[10]   # string not have that index

IndexError: string index out of range
```

In [ ]:
```
s1[-2]
```

backward indexing

In [ ]:
```
s = 'welcome'
s
```

In [ ]:
```
s[-3]
```

In [ ]:
```
s[-10]  # index not exit
```

In [ ]:
```
s[-4]
```

Step indexing

In [ ]:
```
s[2:3:1]
```

In [ ]:
```
s[:9:-1]
```

In [ ]:
```
s[0:10:2]
```

String slicing

In [ ]:
```
s = 'welcometeam'
```

In [ ]:
```
s[2:10]
```

In [ ]:
```
s[:10]
```

In [ ]:
```
s[4:]
```

In [ ]:
```
s[1:10:3]
```

In [ ]:
```
s[::1]
```

In [ ]:
```
s[::2]
```

In [ ]:
```
s[::-1]
```

In [46]:
```
s[::-2]
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[46], line 1
----> 1 s[::-2]

NameError: name 's' is not defined
```

In [ ]:
```
s[:]
```

In [47]:
```
s[::3]
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[47], line 1
----> 1 s[::3]

NameError: name 's' is not defined
```

In [ ]:
```
s[-9:-1:3]
```

forward slicing

In [48]:
```
s = 'welcometeam'
s
```

Out[48]:
```
'welcometeam'
```

In [49]:
```
s[-10:-1:2]
```

Out[49]:
```
'ecmta'
```

In [50]:
```
s[3:10:2]
```

Out[50]:
```
'cmta'
```

In [51]:
```
s = "PYTHON"
print(s[1:5])
```

```
YTHO
```

backward slicing

In [52]:
```
s = 'welcometeam'
s
```

Out[52]:
```
'welcometeam'
```

In [53]:
```
s = "PYTHON"
print(s[::-1])
```

```
NOHTYP
```

In [54]:
```
s = 'welcometeam'
s
```

Out[54]: 'welcometeam'

In [55]:
```
s[-9:-1:3]
```

Out[55]: 'lme'

step slicing

In [56]:
```
s = 'welcometeam'
s
```

Out[56]: 'welcometeam'

In [57]:
```
s[::3]
```

Out[57]: 'wcea'

In [58]:
```
s[::-1]
```

Out[58]: 'maetemoclew'

type casting

In [59]:
```
bool( )
```

Out[59]: False

In [60]:
```
bool(1)
```

Out[60]: True

In [61]:
```
bool(@)   # special characters are not allowed
```

```
  Cell In[61], line 1
    bool(@)   # special characters are not allowed
         ^
SyntaxError: invalid syntax
```

In [62]:
```
bool(1.2)
```

Out[62]: True

In [63]:
```
bool('10')
```

Out[63]: True

In [64]:
```
bool(0)
```

Out[64]: False

In [65]:
```
bool('zero')
```

Out[65]: True

Number System

1.binary number

In [66]:
```
int(0b11001)
```

Out[66]: 25

In [67]:
```
int(0b1110)
```

Out[67]: 14

In [68]:
```
bin(25)
```

Out[68]: '0b11001'

2.Octal number

In [69]:
```
oct(25)
```

Out[69]: '0o31'

In [70]:
```
int(0o31)
```

Out[70]: 25

3.hexadecimal number

In [71]:
```
hex(0)
```

Out[71]: '0x0'

In [72]:
```
hex(10)
```

Out[72]: '0xa'

In [73]:
```
hex(16)
```

Out[73]: '0x10'

In [74]:
```
hex(17)
```

Out[74]: '0x11'

In [75]:
```
int(0x10)
```

Out[75]: 16

In [76]:
```
hex(11)
```

Out[76]: '0xb'

Bitwise Operator

1.Comlement

In [77]:
```
~11
```

Out[77]: -12

In [78]:
```
~45
```

Out[78]: -46

In [79]:
```
~23
```

Out[79]: -24

2.And

In [80]:
```
12&13
```

Out[80]: 12

In [81]:
```
12&13
```

Out[81]: 12

In [82]:
```
12&15
```

Out[82]: 12

In [83]:
```
11&16
```

Out[83]: 0

3. Or

In [84]:
```
12|13
```

Out[84]: 13

In [85]:
```
12|14
```

Out[85]: 14

In [86]:
```
11|12
```

Out[86]: 15

4. XOR

In [87]:
```
12^13
```

Out[87]: 1

In [88]:
```
13^18
```

Out[88]: 31

5. Left Shift

In [89]:
```
12<<1
```

Out[89]: 24

In [90]:
```
12<<2
```

Out[90]: 48

6. Right Shift

In [91]:
```
12>>2
```

Out[91]: 3

In [92]:
```
12>>5
```

Out[92]: 0

math function

package = collection of modules modules = collection of functions framwork = colection of packages tensor = collection of matrix matrix = collection of vectors vector = collection of scalar scalar = single value

In [93]:
```
x = sqrt(25)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[93], line 1
----> 1 x = sqrt(25)

NameError: name 'sqrt' is not defined
```

In [ ]:
```
import math
math.sqrt(25)
```

In [94]:
```
round(math.sqrt(25))
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[94], line 1
----> 1 round(math.sqrt(25))

NameError: name 'math' is not defined
```

In [ ]:
```
pi = 3.14
pi
```

In [95]:
```
pi = 4.17
pi
```

Out[95]:
```
4.17
```

In [96]:
```
math.floor(3.23)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[96], line 1
----> 1 math.floor(3.23)

NameError: name 'math' is not defined
```

In [ ]:
```
math.ceil(3.23)
```

In [97]:
```
math.pow(3,4)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[97], line 1
----> 1 math.pow(3,4)

NameError: name 'math' is not defined
```

In [ ]:
```
import math as m
m.pow(3,4)
```

In [98]:
```
import math as m
print(m.pow(3,2))
print(m.floor(3.2))
print(m.ceil(3.2))
```

```
9.0
3
4
```

In [99]:
```
from math import sqrt, pow, floor, ceil
print(m.pow(3,2))
print(m.sqrt(10))
print(m.floor(3.2))
print(m.ceil(3.2))
```

```
9.0
3.1622776601683795
3
4
```

In [100…

```
from math import* #usin * we can import all function
print(m.pow(3,2))
print(m.sqrt(10))
print(m.floor(3.2))
print(m.ceil(3.2))
```

```
9.0
3.1622776601683795
3
4
```

input function

In [101…

```
x = input()
x
```

Out[101…

```
'23'
```

In [102…

```
x = input()    # input function only get string
y = input()     # it wont understand as arithmatic operator
z = x + y
print(z)
```

```
1224
```

In [103…

```
type(x)
```

Out[103…

```
str
```

In [104…

```
x1 = int(input("Enter first number:"))
x2 = int(input("Enter first number:"))
z = x1 + x2
print(z)
```

```
69
```

In [105…

```
type(x1)
```

Out[105…

```
int
```

In [106…

```
st = input('enter a string')[0]
print(st)
```

```
h
```

In [107…

```
# string with slicing
st = input('enter a string')[3:]
print(st)
```

```
lo
```

In [108…

```
st = input('enter a string')
print(st)
```

```
renuka
```

In [109…

```
st = int(input('enter a string'))  # if user want to convert string into int() t
print(st)  # slicing use string slicing
# eval input read the expression
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[109], line 1
----> 1 st = int(input(                )) # if user want to convert string into
int() type casting
      2 print(st)  # slicing use string slicing
      3 # eval input read the expression

ValueError: invalid literal for int() with base 10: 'pranali'
```

result = eval(input('enter an expr')) print(result)

In [110…

```
#if you want to call any pkg or module use keywords--import or from
```

In [111…

```
import keyword
keyword.kwlist
```

Out[111…

```
['False',
 'None',
 'True',
 'and',
 'as',
 'assert',
 'async',
 'await',
 'break',
 'class',
 'continue',
 'def',
 'del',
 'elif',
 'else',
 'except',
 'finally',
 'for',
 'from',
 'global',
 'if',
 'import',
 'in',
 'is',
 'lambda',
 'nonlocal',
 'not',
 'or',
 'pass',
 'raise',
 'return',
 'try',
 'while',
 'with',
 'yield']
```

In [ ]: